UNIDAD 2: PROGRAMACIÓN CON PHP

1 PHP 8

PHP (PHP Hypertext Pre-Processor).
www.php.net

- Case sensitive
- Separador ;
- Es un lenguaje interpretado.
- Libre y abierto. Gran comunidad.
- Multiplataforma.
- Soporte para múltiples servidores web.
- Soporte para más de 25 SGBD.
- Soporta POO
- Soporte para comunicarse con servicios usando múltiples protocolos.
- Generación de resultados en múltiples formatos.
- Incorpora funciones de comercio electrónico.
- Extensión de los ficheros ***.php**.
- Sencillo para nuevos programadores pero con posibilidades muy avanzadas para programadores profesionales.
- Fichero de configuración: c:\xampp\php\php.ini.

2 DELIMITADORES

```
<?php
    instrucciones;
?>

<?
    instrucciones;
?>
```

3 COMENTARIOS

- //
- /*.....*/
- #

4 VARIABLES

- No hay que declararlas ni especificar el tipo. Se asigna en función del valor que tenga la variable
- Comienzan por $ o _

```php
//Declaración de variables
$edad = 23;
$nombre = "María";
$apellido = 'Martín';
$estatura = 1.60;
$alumno = true;
$Edad = 0;


//Mostrar los tipos de datos
echo gettype($edad).gettype($nombre).gettype($estatura).gettype($alumno);
```

5 GENERACIÓN DE CÓDIGO HTML DESDE PHP

- echo $v1, $v2…: más de un parámetro.
- print($texto): sólo un parámetro.
- printf(textoConFormato,parametros): texto con formato.
- sprintf(textoConFormato,parametros); Idem a printf pero devuelve una cadena. Es útil para no concatenar.

```
echo "<h2>Me llamo $nombre $apellido</h2> y mido $estatura";
echo $edad."<br/>";
echo 'Tengo '. $edad . 'años<br/>';
echo "Tengo $edad años<br/>";
echo "Alumno:$alumno<br/>";
```

6 CADENAS DE CARACTERES

- echo “$variable” se sustituye por el valor de la variable.
- echo ‘$variable’ no se sustituye.
- Secuencias de escape
  - Dentro de ‘’ sólo \\ y \’
  - Dentro de “” no se puede usar \’
- Operador . Para concatenar cadenas

Secuencia	Resultado
\\	muestra una barra invertida.
\'	muestra una comilla simple.
\"	muestra una comilla doble.
\n	muestra un avance de línea (LF o 0x0A (10) en ASCII).
\r	muestra un retorno de carro (CR o 0x0D (13) en ASCII).
\t	muestra un tabulador horizontal (HT o 0x09 (9) en ASCII).
\v	muestra un tabulador vertical (VT o 0x0B (11) en ASCII).
\f	muestra un avance de página (FF o 0x0C (12) en ASCII).
\$	muestra un signo de dólar.

7. TIPOS DE DATOS

- Simples
  - integer: Cualquier número sin decimales. Se pueden representar en formato decimal, octal (comenzando por un 0), o hexadecimal (comenzando por 0x).
  - float: Cualquier número con decimales. Se pueden representar también en notación científica.
  - string: Conjuntos de caracteres delimitados por comillas simples o dobles.
  - boolean: true o false, cualquier valor o 0.
  - null: No hay valor
- Compuestos
  - Array: Primera posición 0
  - object

8 CONVERSIONES DE TIPO

- PHP convierte automáticamente de un tipo a otro cuando lo necesite. Es posible sumar 1 + “10”.
- Es posible realizar una conversión explícita de tipos si se indica entre paréntesis y delante de la variable el tipo al que se desea convertir la variable. (integer)$pi.

```php
$mi_entero = 3;
$mi_real = 2.3;
$resultado = $mi_entero + $mi_real;
// La variable $resultado es de tipo real

$mi_entero = 3;
$mi_real = 2.3;
$resultado = $mi_entero + (int) $mi_real;
// La variable $mi_real se convierte a entero (valor 2) antes de sumarse.
// La variable $resultado es de tipo entero (valor 5)
```

EJERCICIO 1

- Realiza una página PHP que:
  - muestre en una línea tu nombre y apellidos como texto literal.
  - Crea una variable que almacene tu edad y tu población y muéstralo en otra línea.

EJERCICIO 2

- Modifica el programa del ejercicio 1 y añade una lista donde se muestre el tipo de las variables usadas.

EJERCICIO 3

- Crea una página como la que se muestra una nota se define en el código con el formato que se muestra en los ejemplos.

Nota: 3,5

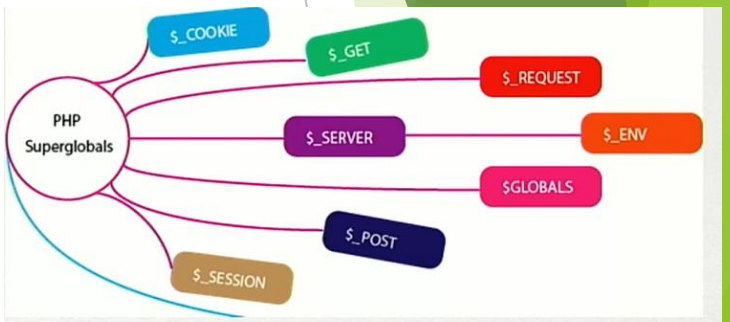
Calificación: Suspenso

Nota: 8,5

Calificación: Notable

9 ÁMBITO DE VARIABLES

- Variables son **locales** en el módulo que se definen.
- Variables **globales**:
  - global $variable: para usar una variable global dentro de una función.
- Variables **estáticas**:
  - static $variable: para que el valor de la variable se mantenga en las siguientes llamadas. Deben ser inicializadas.
- Variables **Predefinidas**
  - $_SERVER: información del servidor y del entorno de ejecución.
  - $_GET, $_POST y $_COOKIE contienen las variables que se han pasado al guión actual utilizando respectivamente los métodos GET (parámetros en la URL), HTTP POST y Cookies HTTP.
  - $_REQUEST junta en uno solo el contenido de los tres arrays anteriores, $_GET, $_POST y $_COOKIE.
  - $_ENV contiene las variables que se puedan haber pasado a PHP desde el entorno en que se ejecuta.
  - $_FILES contiene los ficheros que se puedan haber subido al servidor utilizando el método POST.
  - $_SESSION contiene las variables de sesión disponibles para el guión actual.
- Variables de variables
  - $var1="var2":;
  - $var2="3";
  - echo $$var1;

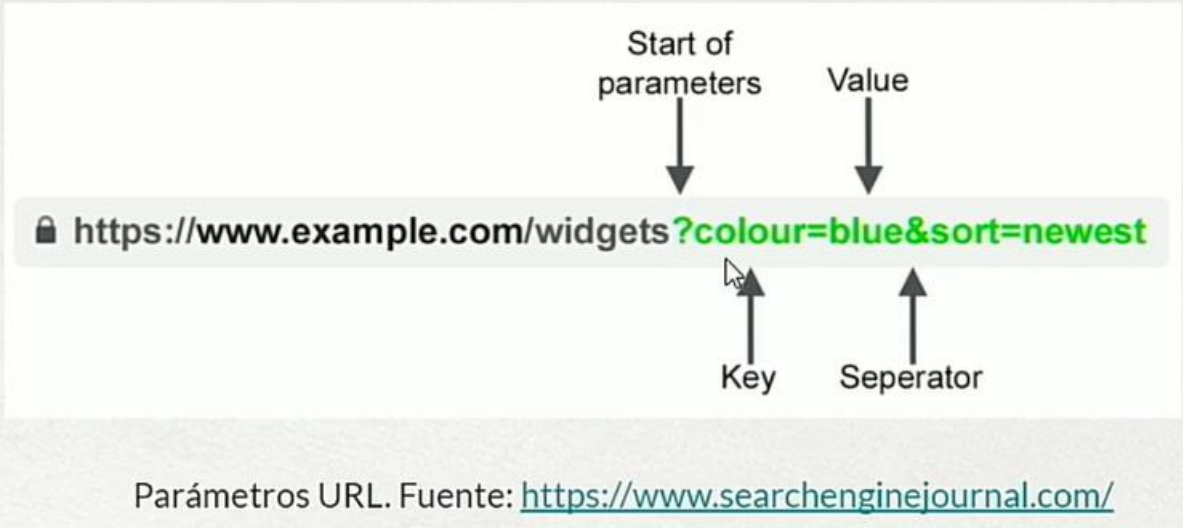


Variables. Fuente: https://ayuda.lalomarquez.net/

10 CONSTANTES

- **const NOMBRE_CONSTANTE = valor** ó **define('NOMBRE_CONSTANTE', valor)**
- Por convenio los nombres se escriben en mayúsculas
- Constantes Predefinidas: PHP_VERSION, **PHP_EOL, PHP_INT_SIZE…**

11 PASO DE VARIABLES POR URL => GET



Parámetros URL. Fuente: https://www.searchenginejournal.com/

Acceso a los parámetros

```php
<?php

    echo "<br/>Color: ".$_GET["color"];
    echo "<br/>Orden: ".$_GET["sort"];

?>
```

Resultado

```
blue
newest
```

EJERCICIO 4

- Crea una página web llamada ejercicio4.php que muestre una lista con tres nombres y sus edades:
  - Ana: 15
  - David: 19
  - Andrés: 23
- Cada nombre de la lista será un enlace a la página a la página ejercicio4Detalle.php que muestre el nombre y la edad del elemento seleccionado. Por ejemplo, si pulso en David, debería mostrarse:

Nombre: David

Edad: 19

12 EXPRESIONES VACÍAS

- No tiene valor
- Si se hace un echo, aparecen en blanco

```
$cadenaVacia="";
$nulo=null;
$falso = false;
```

13 OPERADORES

- Aritméticos: + - * / % **(potencia: 2^3 -> 2**3)
- Asignación: = += -= *= /= %= .=
- Cadenas: .
- Incremento y decremento: ++ -- (echo $a++ echo ++$a)
- Comparación: == > >= < <= != <>
  - Idéntico (mismo valor y mismo tipo): === !==
- Lógicos:
  - and &&
  - or ||
  - !
  - xor (uno de los dos es cierto true xor true devuelve falso)

14 ESTRUCTURAS DE CONTROL SELECTIVAS

```php
if ($a < $b){
    print "a es menor que b";
}
elseif ($a > $b){
    print "a es mayor que b";
}
else{
    print "a es igual a b";
}
```

```php
switch ($a) {
    case 0:
        print "a vale 0";
        break;
    case 1:
        print "a vale 1";
        break;
    default:
        print "a no vale 0 ni 1";
}
```

OPERADOR TERNARIO

```php
$a = 6;
$b=3;
$valor = $a<$b?"a es menor que b":"a es mayor o igual que b";
echo $valor;
```

OPERADOR DE FUSIÓN DE NULL (null coalescing operator)

```php
$valor = $variable ?? $valorPorDefecto;
```

```php
if (isset($variable)) {
    $valor = $variable;
} else {
    $valor = $valorPorDefecto;
}
```

15 ESTRUCTURAS DE CONTROL ITERATIVAS - BUCLES

```php
for($a=1;$a<=10;$a++){
    echo $a;
}
```

```php
//foeach
$datos = array(1,2,3);
foreach ($datos as $valor){
    echo "<br/>$valor";
}
$datosPersona = array("Nombre"=>"Ana","Apellidos"=>"Díaz","Edad"=>20);
foreach ($datosPersona as $clave=>$valor){
    echo "<br/>$clave:$valor";
}
```

```php
while($a<=10){
    echo $a;
    $a++;
}
```

```php
do{
    echo $a;
    $a++;
}while($a<=10);
```

16 SENTENCIAS BREAK Y CONTINUE

- **break** finaliza la ejecución de un bucle en el momento que se ejecuta
- **continue** obliga a que se ejecute la siguiente iteración del bucle
- En bucles anidados se puede añadir un número para indicar sobre cuántas sentencias de control deben finalizar o continuar. (break 2; continue 3;)

EJERCICIO 5

- Mostrar una tabla con las tablas de multiplicar del 1 al 10. El color de las filas debe alternarse entre gris y blanco.

1	2	3	4	5	6	7	8	9	10
2	4	6	8	10	12	14	16	18	20
3	6	9	12	15	18	21	24	27	30
4	8	12	16	20	24	28	32	36	40
5	10	15	20	25	30	35	40	45	50
6	12	18	24	30	36	42	48	54	60
7	14	21	28	35	42	49	56	63	70
8	16	24	32	40	48	56	64	72	80
9	18	27	36	45	54	63	72	81	90
10	20	30	40	50	60	70	80	90	100

EJERCICIO 6

- Hacer un programa que sume, reste, multiplique, divida, calcule el módulo y la potencia de dos variables que almacenan dos números enteros distintos que se generan aleatoriamente (entre 0 y 10). Se debe chequear que las variables son números enteros y que no hay división por 0 e informar del error en caso de que no lo sean.

```
La suma de 6+4 es: 10
La resta de 6-4 es: 2
La multiplicación de 6*4 es: 24
La división de 6/4 es: 1.5
El resto de 6%4 es: 2
La potencia de 6 ^ 4 es: 1296
```

EJERCICIO 7

- Generar tres números entre 1 y 10 de manera aleatoria (función **rand(0,10)**) mostrarlos ordenados de menor a mayor. Si hay números iguales se debe indicar.

Los números generados son: 10, 10, 5

El número 10 está repetido

Los números ordenados son: 5, 10, 10

EJERCICIO 8

- Realiza una página que genere y muestre un número aleatorio entre 0 y 100. Mostrará el nº generado, si el nº generado es menor que 50 se repetirá el proceso, pero y si no se finalizará.

17 DEFINICIÓN DE ARRAYS

Clave	0	1	2	3	4	5
Valor	"Carlos"	"López"			20	1.70

- Escalares
  - Índice es un valor entero empezando en 0

nombre	ape1	ape2	fechan	edad	altura
"Carlos"	"López"			20	1.70

- Asociativos
  - Índice es un string

```
//Definición de arrays
$diasSemana = array("Lunes","Martes","Miércoles");
//Definición corta
$meses = ["Enero","Febrero","Marzo"];
//Con los elementos vacíos
$sinDatos = array(3);
//Array Vacío
$arrayVacio =  array();
```

Pueden almacenar elementos de diferentes tipos

18 ACCESO A LOS ELEMENTOS DE UN ARRAY

```php
//Acceso
echo $diasSemana[1];
echo "<br/>";
echo $meses[0];
```

```php
//Añadir elementos
$diasSemana[] = "Jueves";
echo "<br/>Añdiendo elemento:<br/>";
var_dump($diasSemana);

$sinDatos[0] = "Ana";
$sinDatos[1] = "Martín";
$sinDatos[2] = 1.60;
```

```php
//Función var_dump muestra variables de tipos de
//datos compuesto de una forma estructurada
var_dump($diasSemana);
echo "<br/>";
var_dump($meses);
```

```php
//Son dinámicos
$sinDatos[3] = "profesor";
echo "<br/>";
var_dump($sinDatos);

$arrayVacio[0] = "Ana";
echo "<br/>";
var_dump($arrayVacio);
```

19 ARRAYS ASOCIATIVOS

```php
// Arrays Asociativos
$info = array("nombre"=>"Ana", "apellidos"=>"Martín","estatura"=>1.60);
echo "<br/>";
echo "Nombre:".$info["nombre"]." Apellidos".$info["apellidos"];
echo "<br/>";
var_dump($info);
//Añadir elemento
$info["telefono"]="111222";
echo "<br/>Añdiendo elemento:<br/>";
var_dump($info);
```

EJERCICIO 9

- Crea y muestra un array con los planetas del sistema solar.

EJERCICIO 10

- Crear, rellenar y mostrar dos arrays, uno escalar y otro asociativo, tal y como se muestran en la siguiente imagen. Para mostrar el número de elementos de un array usa la función sizeof($array).

$array1: Array Escalar - Número de elementos 6

0	1	2	3	4	5
Cougar	Ford		2500	V6	182

$array2: Array Asociativo - Número de elementos 6

Modelo	Marca	fecha	CC	Motor	Potencia
Cougar	Ford		2500	V6	182

EJERCICIO 11

- Crea y muestra un array con los 10 primeros números de la serie x^2.

EJERCICIO 12

- Crea un array como el que se muestra a continuación. Recorrer el array y muestra de qué tipo es cada celda del mismo. Para el array asociativo, se debe mostrar todo su contenido (Usa la instrucción foreach para recorres este tipo de array):

2345.65	"Carlos"	34	Nombre / "María" — Edad / 19	True

```
Posición 0:double - Valor:2345.65
Posición 1:string - Valor:Carlos
Posición 2:integer - Valor:34
Posición 3:array - Valor:
=> Nombre:María
=> Edad:19
Posición 4:boolean - Valor:1
```

```
foreach (expresión_array as $valor)
    sentencias
foreach (expresión_array as $clave => $valor)
    sentencias
```

20 DEFINICIÓN DE FUNCIONES

- Deben definirse antes de ser llamadas.
- Admiten **return** si no se pone, devuelven **null**.
- Argumentos
  - Por valor: Opción por defecto
  - Por referencia. &$param
  - Opcionales: $param=valor. Se ponen al final.

```
function precio_con_iva(&$precio, $iva=0.18) {
    $precio *= (1 + $iva);
}
$precio = 10;
print "El precio con IVA es ".$precio_con_iva($precio);
```

```
function precio_con_iva() {
    global $precio;
    $precio_iva = $precio * 1.18;
    print "El precio con IVA es ".$precio_iva;
}

$precio = 10;
precio_con_iva();
```

```
function precio_con_iva($precio) {
    return $precio * 1.18;
}
$precio = 10;
$precio_iva = precio_con_iva($precio);
print "El precio con IVA es ".$precio_iva;
```

21 FUNCIONES DE TIPOS DE DATOS

- **Gettype:** obtiene el tipo de la variable que se le pasa como parámetro y devuelve una cadena de texto, que puede *ser array, boolean, double, integer, object, string, null, resource o unknown type*.
- *is_array(), is_bool(), is_float(),is_integer(), is_null(), is_numeric(), is_object(), is_resource(), is_scalar() e is_string().*
- **Devuelven true si la variable es del tipo indicado.**
- **settype($variable,"tipo"):** establecer el tipo de una variable.
- **isset($variable)**: Comprueba si una variable está definida y no es null.
- **empty($variable)**: Comprueba si una variable está vacía. Devuelve true si la variable no existe o está vacía.
- **unset($variable)**: Destruye un variable y libera memoria.

22 FUNCIONES DE FECHA Y HORA

- time(): Instante actual en formato Unix (como int).
- string date(string formato, int fecha_hora): Parámetro fecha_hora opcional, por defecto la fecha y hora actual según php.ini.
- Para establecer la zona horaria:
  - **date_default_timezone_set('Europe/Madrid')**
- getdate(): Devuelve un array con la fecha y hora actual.
- Caracteres especiales de formato
- strtotime(texto): Convertir texto en fecha
  - strtotime("2022-07-11");

Carácter	Resultado
d	día del mes con dos dígitos.
j	día del mes con uno o dos dígitos (sin ceros iniciales).
z	día del año, comenzando por el cero (0 = 1 de enero).
N	día de la semana (1 = lunes, ..., 7 = domingo).
w	día de la semana (0 = domingo, ..., 6 = sábado).
l	texto del día de la semana, en inglés (Monday, ..., Sunday).
D	texto del día de la semana, solo tres letras, en inglés (Mon, ..., Sun).
W	número de la semana del año.
m	número del mes con dos dígitos.
n	número del mes con uno o dos dígitos (sin ceros iniciales).
t	número de días que tiene el mes.
F	texto del día del mes, en inglés (January, ..., December).
M	texto del día del mes, solo tres letras, en inglés (Jan, ..., Dec).
Y	número del año.
y	dos últimos dígitos del número del año.
L	1 si el año es bisiesto, 0 si no lo es.
h	formato de 12 horas, siempre con dos dígitos.
H	formato de 24 horas, siempre con dos dígitos.
g	formato de 12 horas, con uno o dos dígitos (sin ceros iniciales).
G	formato de 24 horas, con uno o dos dígitos (sin ceros iniciales).
i	minutos, siempre con dos dígitos.
s	segundos, siempre con dos dígitos.
u	microsegundos.
a	am o pm, en minúsculas.
A	AM o PM, en mayúsculas.
r	fecha entera con formato RFC 2822.

EJERCICIO 13

- Mostrar en castellano el día de la semana de la fecha del momento con el formato Lunes, 28 de Septiembre de 2023.

23 FUNCIONES DE ARRAYS

- is_array($variable)
- count($array)
- sizeof($array)
- $claves=array_keys($array)
- $valores=array_values($array)
- array_key_exists($clave,$array)
- in_array($valor, $array): Devuelve true/false
- array_search($valor, $array): Devuelve la clave si lo encuentra.
- array_merge($array1,$array2): Devuelve un array con los elementos de los arrays que se pasan por parámetro.

- Ordenación
  - sort($array)/rsort($array): Ordena de mayor a menor o de menor a mayor. No mantiene los índices.
  - asort($array): Es igual a sort pero mantiene los índices
  - ksort($array): Ordena por los índices. Útil en arrays asociativos.
  - Shuffle($array): Ordena de forma aleatoria.
- Modificación
  - array_ shift($array): Devuelve y Elimina el primer elemento del array
  - array_ unshift($array,$elemento): Inserta un elemento al principio del array
  - array_pop(($array): Devuelve y Elimina el último elemento del array
  - array_push($array,$elemento): Inserta un elemento al final del array

24 FUNCIONES DE CADENAS

- Visualización
  - echo cadena
  - print(cadena)
  - printf(formato, dato1, dato2,…).
  - sprintf(formato, dato1, dato2,…)
- Contenido
  - strlen(cadena)
- Búsqueda
  - Strstr(cadena, cadenaBusq) o Strchr(cadena, cadenaBusq)
  - Strrchr(cadena, caracterBusq)
  - Stristr(cadena, cadenaBusq)
  - Strpos(cadena1, cadena2, desp)
  - Strrpos(cadena, carácter)
- Comparación
  - Strcmp
  - Strcasecmp
  - Strnncmp
  - Strnatcmp
  - Strnatcasecmp
- Relleno
  - Str_pad
- Conversión
  - Strtolower
  - Strtoupper
  - Ucfirst
  - Ucwords
- Limpieza
  - Chop
  - Retrim
  - Ltrim
  - Trim
- División
  - Strtok
  - Chunk_split
  - Split
  - Explode
  - Implode
- Subcadenas
  - Substr
  - Substr_replace
  - Str_replace
  - Strtr

25 INCLUDE/REQUIRE

- Permiten incrustar un archivo dentro de otro
- La diferencia entre una y otra es el tipo de error que generan si no se encuentra el archivo a incrustar:
  - Include genera un error de tipo warning
  - Require genera un error de tipo error.
- Ambas tienen su sentencia correspondiente para que solamente se incruste el código una vez
  - include_once
  - require_once

26 FORMULARIOS

- <form **action**="" **method**="" **enctype**="" **accept-charset**="">
  - **action**: nombre del script php
  - **method**: Get o Post
  - **enctype**="multipart/form-data“ para subir ficheros
- Arrays asociativos
  - **$_GET**
  - **$_POST**
  - **$_REQUEST**
- El formulario se envía al script al pulsar en el botón que realiza el submit. La variable $_SERVER["REQUEST_METHOD"] almacena el método desde el que se llama al script indicado en el atributo action. Es una buena práctica comprobar que ese método coincide con el que se ha puesto en el atributo method del formulario para procesar los datos del formulario.
- Acceder a los campos del formulario desde el script php:
  - Nombre de los campos del formulario serán las claves en $_GET o $_POST
  - Los valores de los elementos de $_GET o $_POST serán los values de los campos del formulario.
  - Para los select de opción múltiple y también para los checkbox, en el name hay que indicar que puede haber un array name=“nombre[]”

27 PROCESAMIENTO DEL FORMULARIO

- Validaciones de campos de un formulario `¿se hacen tanto en cliente como en servidor ?
  - Man in de middel
  - **Ataques de Cross-Site Scripting (XSS):** Inyectar código HTML o JavaScript para que se ejecute en la página.
- La página que procesa los datos es también la página que genera el formulario indicando los errores si los hay.
- Si hay errores, se rellenan los datos correctos y se indican los incorrectos y el motivo.
- Para distinguir si el formulario envía datos o no:
  - Chequear si se ha pulsado el botón que hace el submit: **isset($_POST(“Enviar”))**
  - **Chequear si datos viene rellenos !empty($_POST[“nombre”])**
  - **Chequear si existe en $_POST: isset($_POST[“modulos”])**

27.1 FUNCIONES DE VALIDACIÓN DE DATOS DE ENTRADA

- htmlspecialchars(cadena, flags, codificación): Convierte caracteres especiales en entidades HTML.
  - Cadena:el texto que quieres tratar.
  - Flags:
    - ENT_QUOTES → convierte comillas simples y dobles.
    - ENT_HTML5 → usa entidades HTML5.
  - codificación → normalmente "UTF-8".

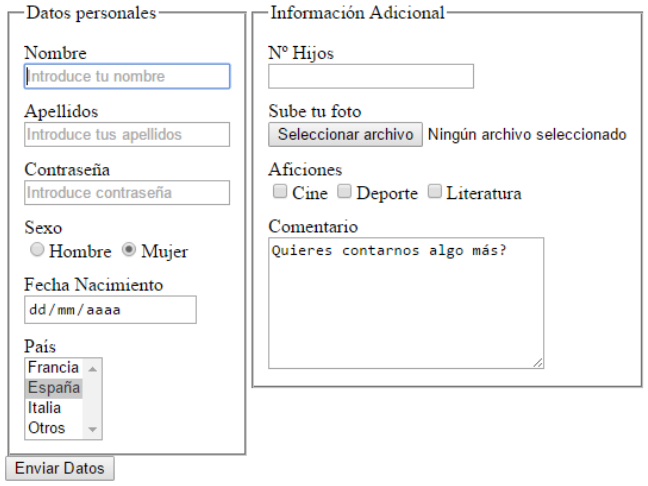
```php
<?php
$comentario = "<script>alert('Hack');</script>";
echo $comentario; //Se ejecuta el alert ya es código javaescript
echo htmlspecialchars($comentario, ENT_HTML5, 'UTF-8'); //No es javascritp

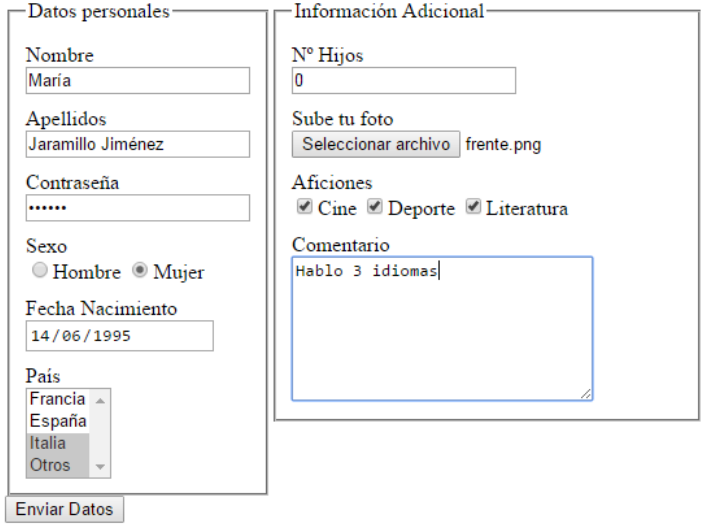
?>
```

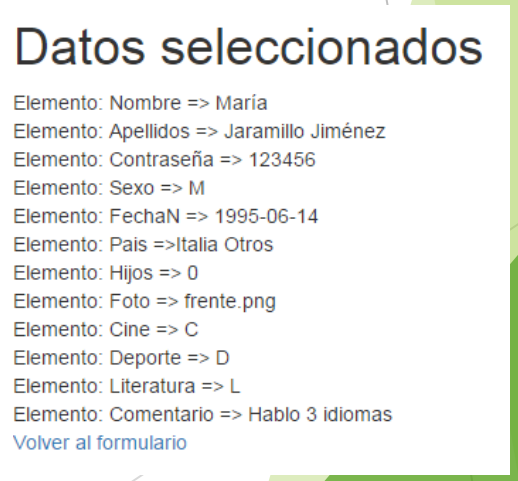
```
&lt;script&gt;alert('Hack');&lt;/script&gt;
```

EJERCICIO 14

- Mostrar la información de cada uno de los elementos del siguiente formulario. Crea dos ficheros, uno para el formulario y otro para tratar los datos:
- Formulario sin datos: El campo país es una lista de opción múltiple. El campo Nº de Hijos es una lista de selección simple con valores 0, 1, 2, 3 , 4 (para 4 o más).







28 ENVIAR FICHERO EN FORMULARIO

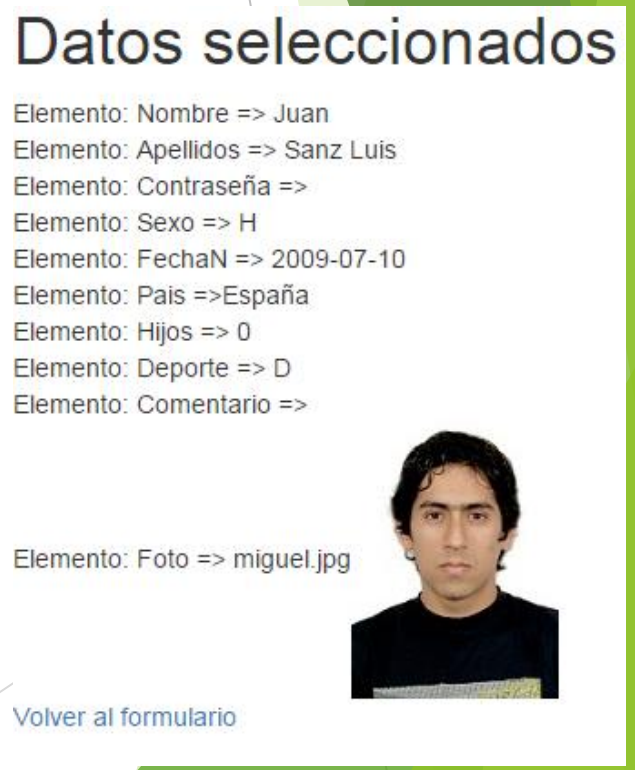
- enctype="multipart/form-data“
- $_FILES
  - $_FILES['fichero_usuario']['name']: El nombre original del fichero en la máquina del cliente.
  - $_FILES['fichero_usuario']['type']: El tipo MIME del fichero.
  - $_FILES['fichero_usuario']['size']: El tamaño, en bytes, del fichero subido.
  - $_FILES['fichero_usuario']['tmp_name']: El nombre temporal del fichero en el cual se almacena el fichero subido en el servidor.
  - $_FILES['fichero_usuario']['error']: El código de error asociado a esta subida

29 SUBIR FICHERO AL SERVIDOR

```
if(isset($_FILES["Foto"])){
    //Establecer Carpeta destino en el servidor a una carpeta  llamada fotos
    //Es una ruta relativa
    $dest="fotos/".$_FILES["Foto"]["name"];
    //Obtener el Fichero temporal subido
    $f=$_FILES["Foto"]["tmp_name"];
    //Copiar el fichero temporal a la carpeta del servidor
    if(move_uploaded_file($f, $dest)){
      echo “Fichero subido con éxito”;
    }
    else echo "Error al subir la foto";
}
```

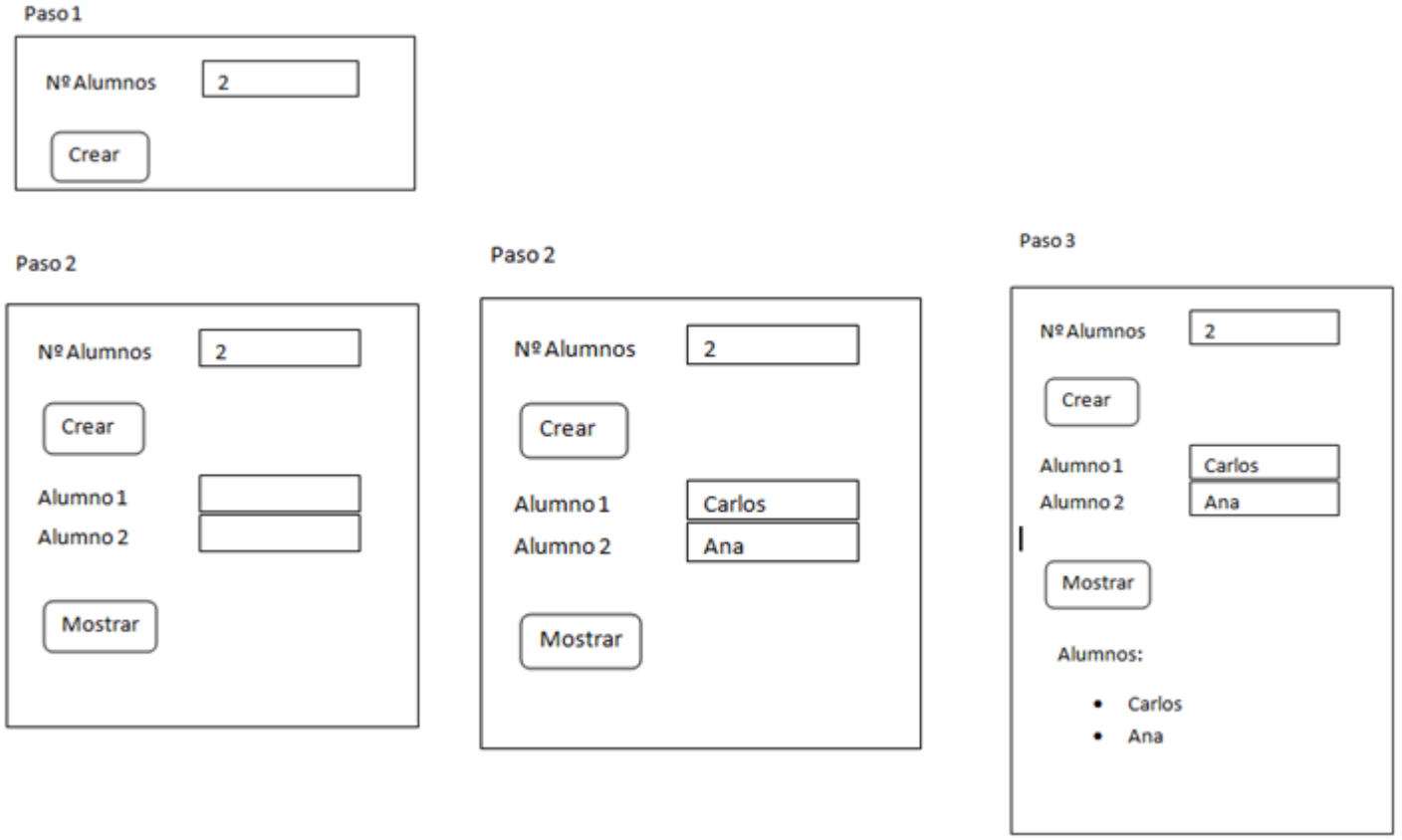
EJERCICIO 15

- Haz una copia del ejercicio del apartado anterior y modifícalo para que la página que trata los datos:
  - Chequee que el nombre, los apellidos y la fecha de nacimiento siempre se rellenen y en caso de que no se haga, se informe del error.
  - Muestre la foto subida por el usuario. El fichero debe almacenarse en la carpeta fotos del sitio web que deberás crear previamente.



EJERCICIO 16

- Formulario en tres pasos:



30 FICHEROS

- Apertura
  - descriptor **fopen** (nombre , modo [, usar_include_path])
  - En el modo a+ cualquier información se añade siempre al final
- Cierre
  - fclose(descriptor)
- Recorrido
  - fgets(decriptor[, num_caracteres]): Lee una línea o los caracteres indicados.
  - rewind (descriptor): Sitúa el puntero de lectura/escritura al principio del fichero.
  - fseek (descriptor. Num_pos[, desde_donde]) : Desplaza la posición del puntero de lectura/escritura Num_pos posiciones. Desde_donde SEEK_SET, SEEK_CUR y SEEK_END.
  - ftell (descriptor): Devuelve la posición del puntero.
  - feof(descriptor): Comprueba si el puntero a un archivo está al final del archivo.
  - fwrite o fputs(descriptor,cadena): Escribe la cadena en el fichero. Escribir fin de línea PHP_EOL.
- Otras
  - file(nombre_fichero, FILE_IGNORE_NEW_LINES): Transfiere un fichero completo a un array. No transfiere el salto de línea a cada elemento del array.
  - readfile(nombre_fichero): Lee el contenido de un fichero y lo muestra por la salida estándar. Devuelve el total de caracteres leídos.
  - file_exists(nombre_fichero) - Comprueba si existe un fichero o directorio.
  - is_readable(nombre_fichero) - Indica si un fichero existe y es legible.
  - is_writedable(nombre_fichero) - Indica si un fichero existe y es legible.
  - is_file (nombre_fichero): Devuelve verdadero si el tipo del fichero es un fichero .
  - is_dir (nombre_fichero): Devuelve verdadero si el nombre del fichero es un directorio.
  - is_executable (nombre_fichero): Devuelve verdadero si el nombre del fichero pasado como argumento tiene perrnisos de ejecución (Unix) o es un fichero ejecutable (extensión exe. com o bat).
  - filesize(nombre_fichero): Devuelve el tamaño del fichero
  - filemtime(nombre_fichero): Devuelve la fecha de la última modificación del fichero.

31 MODOS DE APERTURA (Ampliación)

'r'	Apertura para sólo lectura; coloca el puntero al fichero al principio del fichero.
'r+'	Apertura para lectura y escritura; coloca el puntero al fichero al principio del fichero.
'w'	Apertura para sólo escritura; coloca el puntero al fichero al principio del fichero y trunca el fichero a longitud cero. Si el fichero no existe se intenta crear.
'w+'	Apertura para lectura y escritura; coloca el puntero al fichero al principio del fichero y trunca el fichero a longitud cero. Si el fichero no existe se intenta crear.
'a'	Apertura para sólo escritura; coloca el puntero del fichero al final del mismo. Si el fichero no existe, se intenta crear. En este modo, fseek() solamente afecta a la posición de lectura; las lecturas siempre son pospuestas.
'a+'	Apertura para lectura y escritura; coloca el puntero del fichero al final del mismo. Si el fichero no existe, se intenta crear. En este modo, fseek() solamente afecta a la posición de lectura; las lecturas siempre son pospuestas.
'x'	Creación y apertura para sólo escritura; coloca el puntero del fichero al principio del mismo. Si el fichero ya existe, la llamada a **fopen()** fallará devolviendo **FALSE** y generando un error de nivel **E_WARNING**. Si el fichero no exite se intenta crear. Esto es equivalente a especificar las banderas *O_EXCL|O_CREAT* para la llamada al sistema de *open(2)* subyacente.
'x+'	Creación y apertura para lectura y escritura; de otro modo tiene el mismo comportamiento que *'x'*.

32 OPERACIONES CON FICHEROS (a nivel externo) (Ampliación)

- copy (origen, destino): Copia un fichero destino. Si no ha habido ningún error, devuelve TRUE.
- rename (nombre_original, nombre_final) : Cambia el nombre a un fichero o directorio.
- unlink (nom_fichero): Borra un fichero.

33 OPERACIONES CON CARPETAS (Ampliación)

- opendir (nombre): Abre el directorio. Devuelve un descriptor de directorio.
- readdir (descriptor): Lee una entrada del directorio a partir del descriptor.
- closedir (descriptor): Cierra el directorio indicador por el descriptor.
- rewinddir (descriptor): Sitúa el puntero de lectura al principio del directorio.
- getcwd (): Devuelve el directorio actual de trabajo (también llamado activo) del script.
- chdir(nuevo_dir): Establece nuevo_dir como nuevo directorio de trabajo. Devuelve FALSE en caso de error.
- mkdir (nombre_dir, permisos): Creamos el directorio nombre_dir con los permisos indicados en el segundo parámetro
- rmdir (nombre_dir): Borra el directorio nombre_dir.

34 REDIRECCIONAR A OTRA PÁGINA

```php
<?php
header('Location: pagina.php');
?>
```

35 CONTROL DE EXCEPCIONES

- Clases
  - Exception
  - Error
  - Trowable: De ella derivan tanto Exception como Error por lo que se puede usar para capturar las dos anteriores.
- Excepciones predefinidas

```
try {


 } catch (Exception $e) {
     echo $e->getMessage();
 }
 finally {

 }
```

```
try {


 } catch (Error $e) {
     echo $e->getMessage();
 }
 finally {

 }
```

```
try {


 } catch (Throwable $e) {
     echo $e->getMessage();
 }
 finally {

 }
```

```
Error
  ArithmeticError
    DivisionByZeroError
  AssertionError
  ParseError
  TypeError
    ArgumentCountError
Exception
  ClosedGeneratorException
  DOMException
  ErrorException
  IntlException
  LogicException
    BadFunctionCallException
      BadMethodCallException
    DomainException
    InvalidArgumentException
    LengthException
    OutOfRangeException
  PharException
  ReflectionException
  RuntimeException
    OutOfBoundsException
    OverflowException
    PDOException
    RangeException
    UnderflowException
    UnexpectedValueException
  SodiumException
```

36 POO- DEFINICIÓN DE CLASE

```php
<?php
    // Fichero empleado.inc - Definición de clase empleado
    class empleado{
        var $nombre;
        var $sueldo;

        function mostrar(){
            echo "<hr/>";
            echo "Nombre: ", $this->nombre, "<br/>";
            echo "Sueldo: ", $this->sueldo, "<br/>";
            echo "<hr/>";
        }
    }
?>
```

37 POO – CREACIÓN DE UN OBJETO

```
<!-- Fichero empleado-php - Declaración de un objeto de la clase empleado -->
<!DOCTYPE html>
<html>
<head>
<?php
header ( 'Content-Type: text/html; charset=ISO-8859-1' )?>
<title>Solución ejercicios estructuras de control</title>
</head>
<body>
    <?php
        include_once "empleado.inc";

        //Creación del objeto
        $conserje = new empleado();

        //Acceso a atributos
        $conserje->nombre="José";
        $conserje->sueldo = 1500;

        //Acceso a métodos
        $conserje->mostrar();

    ?>
</body>
</html>
```

38 POO - CONSTRUCTOR

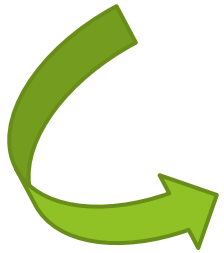
```php
<?php
    // Fichero empleado.inc - Definición de clase emp
    class empleado{
        var $nombre;        // Atributo público
        private $sueldo;    // Atributo privado

        function mostrar(){
            echo "Nombre: ", $this->nombre, "<br/>";
            echo "Sueldo: ", $this->sueldo;

        }

        function __construct($nombre){
            $this->nombre=$nombre;
            $this->sueldo=1000;

        }
    }
?>
```



```php
<!-- Fichero empleado-php - Declaración de un o
<!DOCTYPE html>
<html>
<head>
<?php
header ( 'Content-Type: text/html; charset=ISO-
<title>Solución ejercicios estructuras de contr
</head>
<body>
    <?php
        include_once "empleado.inc";

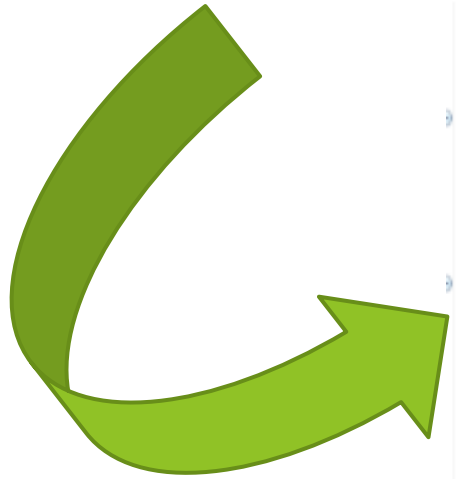
        //Creación del objeto
        $conserje = new empleado("Andrés");

        //Acceso a métodos
        $conserje->mostrar();

    ?>
</body>
</html>
```

39 POO - DESTRUCTOR

```php
function __destruct(){
    echo "<br/>Me han despedido :( :(";
}
```



```php
<!-- Fichero empleado-php - Declaración de u
<!DOCTYPE html>
<html>
<head>
<?php
header ( 'Content-Type: text/html; charset=I
<title>Solución ejercicios estructuras de co
</head>
<body>
    <?php
        include_once "empleado.inc";

        //Creación del objeto
        $conserje = new empleado("Andrés");


        //Acceso a métodos
        $conserje->mostrar();

        //Destrucción de objeto
        unset($conserje);

    ?>
</body>
</html>
```

40 POO - ENCAPSULACIÓN

```php
<?php
    // Fichero empleado.inc - Definición de clase emplea
    class empleado{
        var $nombre;        // Atributo público
        private $sueldo;    // Atributo privado

        function mostrar(){
            echo "Nombre: ", $this->nombre, "<br/>";
            echo "Sueldo: ", $this->sueldo;

        }
        function __construct($nombre){
            $this->nombre=$nombre;
            $this->sueldo=1000;

        }
        function __destruct(){
            echo "<br/>Me han despedido :( :(";
        }
        // Método privado
        private function subir_sueldo($incremento) {
            $this->sueldo+=$incremento;
        }
        function premiar($tipo){
            switch ($tipot){
                case 1: $this->subir_sueldo(100);break;
                case 2: $this->subir_sueldo(200);break;
                case 3: $this->subir_sueldo(300);break;
            }

        }
    }
?>
```

- Públicos
  - var elemento
- Privados: Sólo son accesibles desde la clase.
  - private elemento
- Protegidos: Son accesibles desde la clase y desde sus descendientes.
  - protected elemento
- Constantes de clase
  - Definición: const NOMBRE
  - Acceso: clase::NOMBRE

41 ATRIBUTOS Y MÉTODOS ESTÁTICOS

- Son atributos de la clase, se accede a ellos directamente desde la clase.
- Sintaxis:
  - Declaración: static elemento
  - Uso: **clase::elemento**
- No forman parte de los objetos por eso no se puede usar this->elemento
- Ejemplo: clase que permita saber el número de objetos que hay instanciados de esa clase

```php
<?php
class objeto{
    static private $contador = 0;

    static function mostrar_cantidad(){
        return  "El número de objetos creados de la clase objetos son".objeto::$contador."<br/>";
    }

    function __construct() {
        objeto::$contador++;
        echo "Se ha creado un nuevo objeto<br/>";
    }
    function __destruct() {
            objeto::$contador--;
            echo "Se ha eliminado un objeto<br/>";
    }
}
?>
```

42 POO - HERENCIA

- Una clase puede tener sus propios elementos y los de una clase padre de la que hereda:
  - class clase_hija extends clase_padre{}
- No se permite herencia múltiple. Esto se simula con los interfaces.
- No se llama automáticamente al constructor/destructor de la clase padre, hay que hacerlo de manera explícita
  - parent::__construct()
  - parent::__destruct()

43 POO – CLASES MÉTODOS ABSTRACTOS

- Las clases abstractas no pueden instanciarse. Hay que codificar una subclase.
  - Sintaxis: abstract class clase_abstracta{}
- Los métodos abstractos se tienen que redefinir en la clase hija, no tienen código
  - Sintaxis: abstract function metodo_abstracto(){}

44 POO - CLASES y MÉTODOS FINALES

- Las clases finales no pueden ser heredadas.
  - Sintaxis: final class miclase{}
- Los métodos finales no pueden ser redefinidos en las clases hijas
  - Sintaxis: final function metodo_final(){}

45 POO - INTERFACES

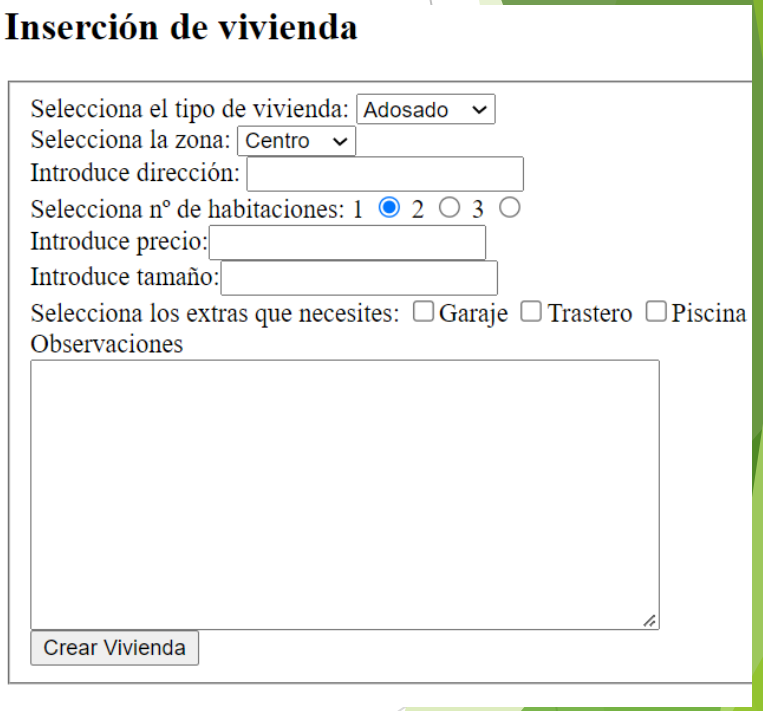
- Es una clase abstracta en la que sólo se declaran métodos.
- No se pueden declarar atributos.
- Sintaxis: interface nombre_interface{}
- Las clases que implementen un interface deben redefinir los métodos.
  - Sintaxis: class miclase implements nombre_interface{}
  - Una clase puede implementar más de un interface (similar al concepto de herencia múltiple)

EJERCICIO 17

- Utilizando POO realiza una página web que permita gestionar las citas de un pequeño centro de manicura.
  - Fecha de la cita
  - Hora de la cita
  - Nombre del cliente
  - Tipo de servicio: Corte señora, corte caballero, tinte, mechas.
- Debes crear la clase Cita y trabajar con ella.
- El formulario debe recoger la información que se detalla a continuación que se almacenará en un fichero de texto separado por ;.
- La página debe mostrar todas las citas que hay en el fichero. Mostrará todos los datos y el tiempo estimado. El tiempo estimado no será un atributo, la clase cita tendrá un método que devolverá el tiempo de esa cita. Los tiempos son: Corte señora->30', corte caballero->15', tinte->120', mechas->180'.

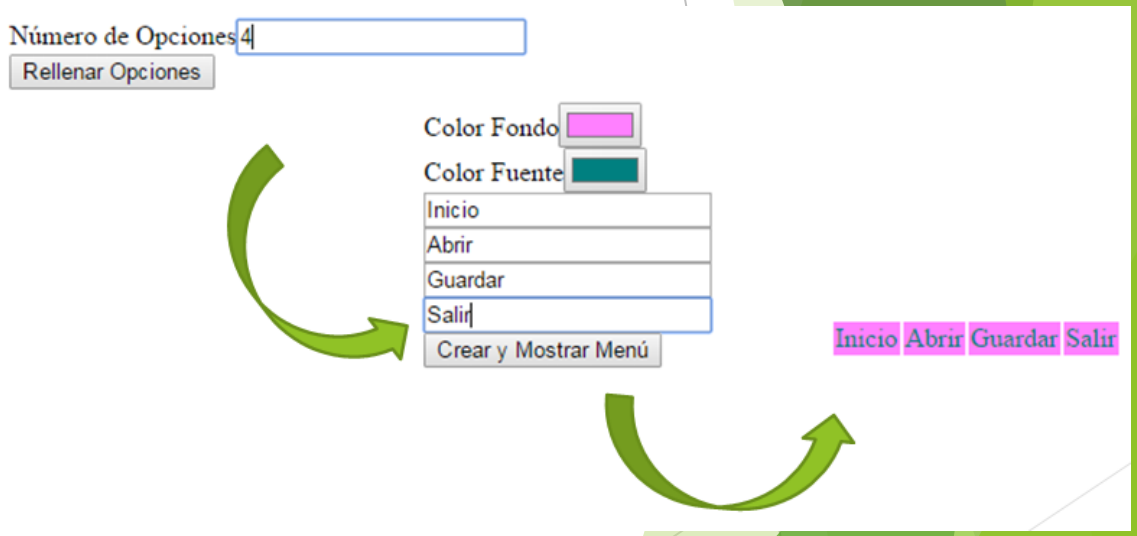
EJERCICIO 18

- Realiza el siguiente formulario que permite crear las viviendas de una inmobiliaria.
- Tipo de vivienda: Adosado, unifamiliar y piso
- Zona: Centro y periferia
- Utiliza para los checkbox un array, al igual que en los selects de opción múltiple. Rellena el value del checkbox.
- Se debe chequear que ningún campo (excepto las observaciones y los extras) están vacíos.
- La viviendas introducidas se almacenará en un fichero de texto separado por ;.
- La página debe mostrar todas las viviendas que hay en el fichero.
- Debes crear la clase Vivienda y trabajar con ella.



EJERCICIO 19

- Utilizando POO realiza una página web que cree un menú en el que el número de opciones del menú y los textos de las opciones sean configurables por el usuario.
- El menú será un objeto con los siguientes atributos:
  - Array de opciones
  - Color de fondo
  - Color del texto
- Tendrá los siguientes métodos:
  - Constructor, que rellenará los atributos con los valores introducidos por el usuario en el formulario.
  - Pintar, que pintará las opciones del menú con los colores correspondientes.



46 NAMESPACES

- Permiten organizar el código y evitar colisiones de nombres (similar a los paquetes de Java)

Definición del NameSpace

```php
<?php
// archivo: Modelos/Usuario.php
namespace App\Modelos;

class Usuario {
    public function saludar() {
        return "Hola desde la clase Usuario.";
    }
}
```

Uso del NameSpace

```php
<?php
// archivo: index.php

// Incluir el archivo donde está la clase Usuario
require 'Modelos/Usuario.php';

// Importar la clase Usuario del namespace App\Modelos
use App\Modelos\Usuario;

$usuario = new Usuario();
echo $usuario->saludar();
```

47 Expresiones regulares en PHP

Todas las expresiones regulares de PHP deben estar delimitadas. El delimitador más utilizado es / aunque también puedes usar #, ~, %, etc.

Los modificadores se escriben después del delimitador de cierre (/patrón/i).

```php
$cadena = "Hola, mi correo es usuario@example.com";
$patron = "/[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}/i";

if (preg_match($patron, $cadena, $coincidencias)) {
    echo "Correo encontrado: " . $coincidencias[0];
} else {
    echo "No se encontró un correo";
}
```

```php
$patron = "/expresion_regular/modificadores";
```

```php
$patron = "/hola/";  // Coincide con la palabra "hola"
```

Modificador	Significado
`i`	No distingue mayúsculas de minúsculas
`m`	Trata los saltos de línea como inicio y fin de línea
`s`	Hace que `.` también coincida con saltos de línea

Metacarácter	Significado	Ejemplo
`.`	Cualquier carácter	`/h.t/` → Coincide con "hat", "hit", "hot"
`\d`	Dígito (`0-9`)	`/\d+/` → Encuentra "123" en "abc 123"
`\w`	Letra, número o `_`	`/\w+/` → Encuentra "Hola" en "Hola mundo"
`\s`	Espacio en blanco	`/\s+/` → Encuentra espacios, tabulaciones o saltos de línea
`^`	Inicio de cadena	`/^Hola/` → Coincide solo si la cadena empieza con "Hola"
`$`	Fin de cadena	`/mundo$/` → Coincide si la cadena termina en "mundo"
`\b`	Límite de palabra	`/\bPHP\b/` → Coincide solo con la palabra "PHP" completa

Cuantificador	Significado	Ejemplo
`*`	0 o más veces	`/a*/` → "aaa", "a", ""
`+`	1 o más veces	`/a+/` → "aaa", "a", pero no ""
`?`	0 o 1 vez (opcional)	`/a?/` → "a" o ""
`{n}`	Exactamente `n` veces	`/\d{3}/` → "123" en "abc 12345"
`{n,}`	`n` o más veces	`/\d{2,}/` → "123" en "abc 123"
`{n,m}`	Entre `n` y `m` veces	`/\d{2,4}/` → "123" en "abc 12345"